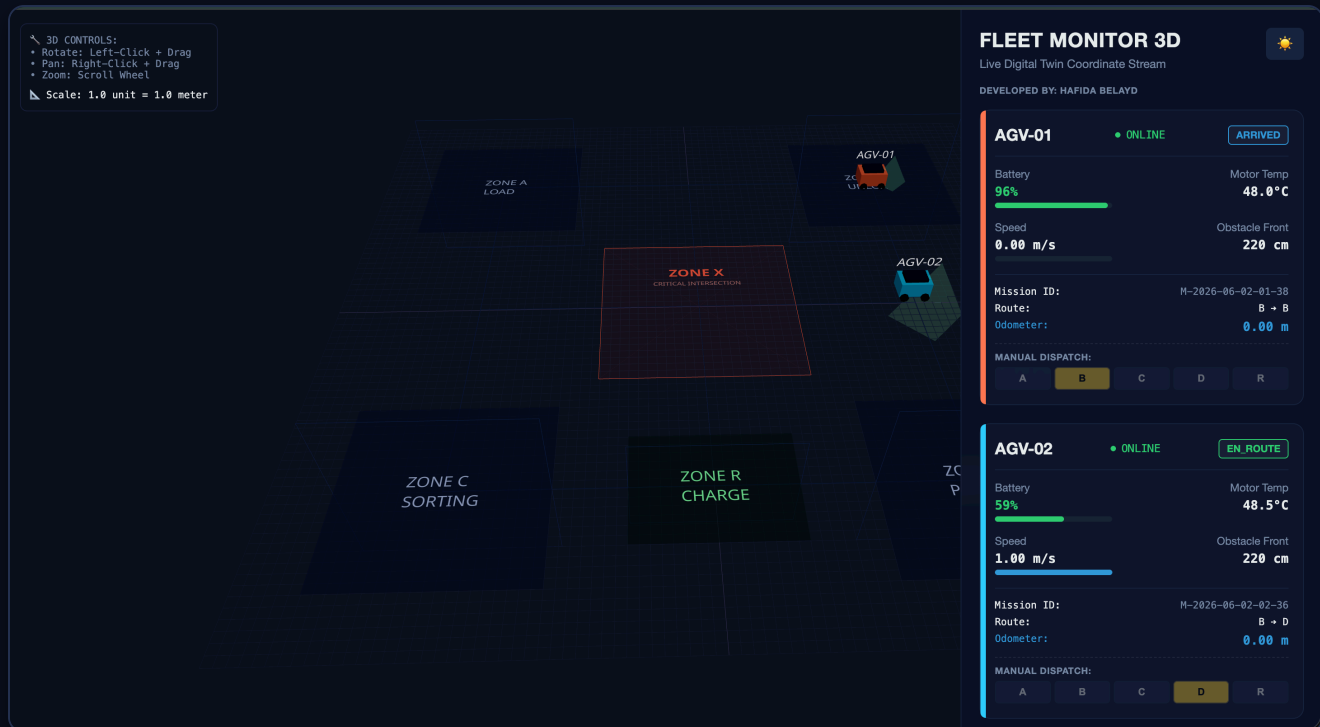


Documentation Jour 3 - Évolution de Sécurité & Dashboard 3D



1. Objectif de l'Étape 3

Projet : Physical AI - ABA Fusion | Auteur : Hafida Belayd



Nouveautés du Jour 3 : Cette journée s'est concentrée sur la fiabilisation du robot physique avec l'ajout de mécanismes d'arrêt d'urgence (Safety Stop) matériels, l'amélioration du traitement des données asynchrones (Non-Blocking), et la consolidation de l'intégration avec le Dashboard 3D React.

Points Clés Abordés

- Implémentation d'un arrêt d'urgence matériel conditionné par la détection du capteur ultrasonique.
- Optimisation de la lecture des capteurs (Multi-échantillonnage et délai de retour d'écho allongé).
- Alignement esthétique de la documentation avec le thème Cyber-Industriel du Dashboard.

2. Intégration du Capteur HC-SR04 et Système d'Arrêt Double (Dual Safety)

Le robot a été équipé d'un capteur à ultrasons **HC-SR04** pour détecter les obstacles frontaux. Pour garantir une sécurité absolue, la logique d'arrêt d'urgence a été implémentée à deux niveaux : matériel (C++) et logiciel (Python).

2.1. Niveau Matériel : C++ (ESP32 / Arduino)

Afin d'éviter toute collision physique en cas de latence réseau, un mécanisme autonome d'arrêt d'urgence est exécuté directement sur le microcontrôleur.

```
const float SAFETY_STOP_CM = 20.0;

bool isObstacleTooClose() {
    return latestDistance > 0.0 && latestDistance <= SAFETY_STOP_CM;
}

void applySafetyStop() {
    if (!currentlyClose) return;

    // Force les moteurs à 0 indépendamment des commandes réseau
    if (currentM1 > 0) setMotor(1, 0);
    if (currentM2 > 0) setMotor(2, 0);
}
```

Fiabilisation du HC-SR04 : Le code C++ utilise une boucle d'échantillonnage avec un timeout de `30000 µs` pour éviter de bloquer le processeur si aucun écho n'est reçu (Non-blocking). Si un obstacle apparaît à moins de 20cm, les moteurs sont coupés instantanément (Hardware Override).

2.2. Niveau Logiciel : Jumeau Numérique (Python)

Les données du capteur HC-SR04 sont envoyées via télémétrie au simulateur Python. Le jumeau numérique analyse cette distance pour mettre à jour son propre état et synchroniser l'affichage 3D.

```
# Dans agv_agent.py
if self.is_digital_twin_active and (0 < self.distance_front_cm <= 20.0):
```

```
self.state = "STOP"
return # Le jumeau numérique refuse d'avancer
```

Analyse Logique Python : Dès que `headless_simulator.py` reçoit la distance du HC-SR04, il met à jour `agv.distance_front_cm`. L'agent Python détecte l'obstacle et passe en état `STOP`. Cela crée une **Sécurité Redondante** : le robot physique s'arrête de lui-même, et le jumeau numérique confirme cet arrêt sur le Dashboard 3D, empêchant le système central de planifier d'autres mouvements tant que l'obstacle est présent.

2.3. Réception Série Non-Bloquante (Serial Polling)

Pour assurer une réactivité maximale du HC-SR04, la réception des commandes moteur via Serial a été transformée en boucle asynchrone (buffer caractère par caractère) au lieu de fonctions bloquantes.

```
while (Serial.available() > 0) {
  char c = Serial.read();
  if (c == '\n') {
    sscanf(serialBuffer.c_str(), "%d,%d,%d,%d", &m1, &m2, &m3, &m4);
    setMotor(1, m1); /* ... */
    serialBuffer = "";
  } else {
    serialBuffer += c;
  }
}
```

Cela garantit que l'ESP32 peut continuer de lire les capteurs ultrasons et de maintenir la connexion MQTT active via `client.loop()`, sans jamais être interrompu par une latence de transmission série.

3. Optimisation de l'Architecture Frontend

L'affichage est désormais parfaitement intégré au Digital Twin en 3D.

Modifications de l'Architecture Frontend :

- **Architecture Headless** : Le jumeau numérique s'exécute à 100% via `headless_simulator.py`, dont le seul rôle est de calculer les mathématiques, les Mutex, les trajectoires (Dijkstra) et d'émettre les états via WebSocket.
- **Dashboard React / Three.js** : Tout l'affichage (véhicules, entrepôt, métriques, télémétrie en direct) est dorénavant géré par le portail Web (Frontend React) pour une expérience hautement immersive et performante.
- **Thématisation (Dark Theme)** : Adoption d'un code couleur industriel nocturne (Deep Navy `#0a0f1d`, textes Ivoire, alertes Terracotta).

4. Diagramme de la Séquence d'Arrêt d'Urgence

Voici l'illustration du flux de contrôle de l'arrêt d'urgence matériel intégré à l'ESP32 :

